

Sumário

1. Introdução:	2
2. Objetivo	3
3. Implementação	3
4. Testes e Resultados	4
5. Análise dos resultados	5
6. Conclusão	5
Referências	6
Anexos	7
Anexo A – Código-fonte ordenadores.h	7
Anexo B – Código-fonte main.c	7
Anexo C – Código-fonte bubble-sort.c	9
Anexo D – Código-fonte merge-sort.c	10
Anexo E – Código-fonte quick-sort.c	11

1. Introdução:

A análise de algoritmos permite avaliar o desempenho de diferentes soluções computacionais de acordo com o tamanho da entrada de dados. Esse tipo de análise é importante porque algoritmos que funcionam bem para pequenas quantidades de dados podem apresentar desempenho inadequado quando a entrada aumenta.

Nesta atividade, foram comparados três algoritmos de ordenação: Bubble Sort, Merge Sort e Quick Sort. O problema tratado consiste em ordenar vetores de números inteiros gerados aleatoriamente e medir o tempo de execução de cada algoritmo para diferentes tamanhos de entrada.

Para realizar a comparação, foram utilizados vetores com 100, 300, 500, 1000 e 10000 elementos. Cada algoritmo recebeu uma cópia do mesmo vetor original, garantindo que todos fossem executados sobre os mesmos dados. Os tempos obtidos foram registrados em milissegundos e utilizados para a construção de uma tabela e de um gráfico comparativo.

GitHub:

Acesso ao repositório: [GitHub – alineop120/n2_at1 ordenadores](#) .

2. Objetivo

O objetivo desta atividade foi medir e comparar o tempo de execução dos algoritmos Bubble Sort, Merge Sort e Quick Sort em vetores aleatórios de diferentes tamanhos.

Além disso, buscou-se observar, na prática, como o crescimento da entrada influencia o desempenho dos algoritmos e relacionar os resultados obtidos com a complexidade assintótica de cada método, representada pela notação Big-O.

3. Implementação

A atividade foi implementada em linguagem C, utilizando o editor Visual Studio Code e o compilador GCC. Como não foram localizados os códigos mencionados no AVA, o grupo implementou os algoritmos Bubble Sort, Merge Sort e Quick Sort, mantendo o objetivo principal da atividade.

O código-fonte foi organizado na pasta src, contendo o arquivo main.c e a subpasta algoritmos, na qual foram separados os arquivos bubble-sort.c, merge-sort.c e quick-sort.c. Essa organização foi adotada para facilitar a leitura, a manutenção e a identificação da implementação de cada algoritmo de ordenação. Também foi utilizado um arquivo ordenadores.h para declarar os protótipos das funções utilizadas pelos arquivos do projeto.

O arquivo main.c ficou responsável pela geração dos vetores, pela criação das cópias das entradas, pela chamada dos algoritmos de ordenação, pela medição dos tempos de execução e pela gravação dos resultados no arquivo resultados.csv, localizado na pasta resultados.

Para medir o tempo de execução, foi utilizada a função clock(), da biblioteca time.h, com conversão do resultado para milissegundos. Os dados gerados foram posteriormente utilizados para a construção da tabela de resultados e do gráfico de colunas.

4. Testes e Resultados

A tabela a seguir apresenta os tempos de execução obtidos em milissegundos para os algoritmos Bubble Sort, Merge Sort e Quick Sort.

Tabela 1 – Tempos de execução dos algoritmos de ordenação em milissegundos

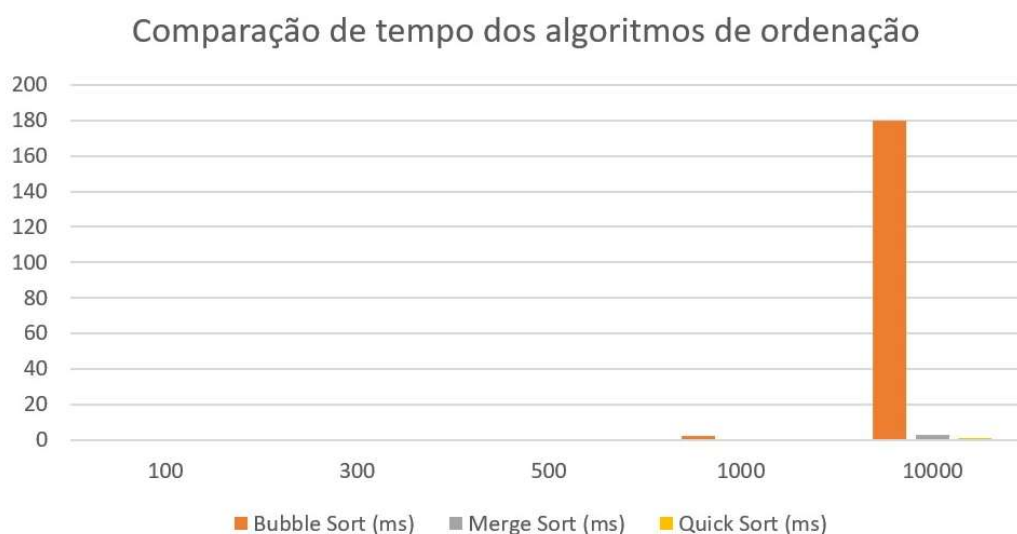
Tamanho do vetor	Bubble Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
100	0	0	0
300	0	0	0
500	0	0	0
1000	2	0	0
10000	180	3	1

Fonte: Elaborado pelo grupo com base nos resultados obtidos na execução do programa em C.

Alguns valores foram registrados como 0 ms porque a execução foi muito rápida para a precisão da função de medição utilizada. Isso ocorreu principalmente nos vetores menores, nos quais os algoritmos executaram em um intervalo de tempo muito pequeno.

A seguir, é apresentado o gráfico de colunas gerado a partir dos tempos obtidos na execução do programa.

Figura 1 – Comparação do tempo de execução dos algoritmos de ordenação



Fonte: Elaborado pelo grupo com base nos resultados obtidos na execução do programa em C.

5. Análise dos resultados

A partir dos resultados obtidos, foi possível observar que o Bubble Sort apresentou o maior crescimento no tempo de execução conforme o tamanho do vetor aumentou. Para vetores pequenos, com 100, 300 e 500 elementos, os tempos registrados foram iguais a 0 ms, pois a execução ocorreu de forma muito rápida.

No vetor com 1000 elementos, o Bubble Sort registrou 2 ms, o Merge Sort registrou 0 ms e o Quick Sort continuou com tempo registrado de 0 ms. Já no vetor com 10000 elementos, a diferença entre os algoritmos ficou mais evidente: o Bubble Sort apresentou tempo de 180 ms, enquanto o Merge Sort apresentou 3 ms e o Quick Sort apresentou 1 ms.

Esse comportamento está relacionado à complexidade dos algoritmos. O Bubble Sort possui complexidade média $O(n^2)$, pois utiliza laços aninhados e realiza muitas comparações entre os elementos do vetor. Dessa forma, seu desempenho piora de maneira significativa quando a quantidade de elementos aumenta.

O Merge Sort possui complexidade $O(n \log n)$, pois utiliza a estratégia de divisão e conquista. Ele divide o vetor em partes menores, ordena essas partes e depois combina os resultados. Por esse motivo, apresentou desempenho melhor que o Bubble Sort em vetores maiores.

O Quick Sort também possui complexidade média $O(n \log n)$. Ele escolhe um pivô e particiona o vetor em elementos menores e maiores que esse pivô. Na execução realizada, o Quick Sort apresentou o menor tempo para o vetor de 10000 elementos, demonstrando desempenho eficiente para a entrada analisada.

Portanto, os resultados confirmam que algoritmos com complexidade $O(n \log n)$, como Merge Sort e Quick Sort, tendem a ser mais eficientes para grandes volumes de dados do que algoritmos de complexidade $O(n^2)$, como o Bubble Sort.

6. Conclusão

A atividade permitiu observar na prática a influência do tamanho da entrada no tempo de execução dos algoritmos de ordenação. Os resultados mostraram que, embora o Bubble Sort seja simples de entender e implementar, ele se torna menos eficiente para grandes volumes de dados devido à sua complexidade $O(n^2)$.

Por outro lado, o Merge Sort e o Quick Sort apresentaram melhor desempenho, pois possuem complexidade média $O(n \log n)$, sendo mais adequados para entradas maiores.

Dessa forma, conclui-se que a escolha do algoritmo é importante para o desempenho de um programa, especialmente quando se trabalha com grandes quantidades de dados. A análise de algoritmos permite prever esse comportamento e escolher soluções computacionalmente mais eficientes.

Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

SERPA, Matheus da Silva et al. **Análise de algoritmos**. Porto Alegre: Grupo A, 2021.

Anexos

Anexo A – Código-fonte ordenadores.h

```
#ifndef ORDENADORES_H
#define ORDENADORES_H

void copiarVetor(int origem[], int destino[], int n);
void gerarVetorAleatorio(int vetor[], int n);

void bubbleSort(int vetor[], int n);
void mergeSort(int vetor[], int inicio, int fim);
void quickSort(int vetor[], int baixo, int alto);

double medirTempoBubble(int vetor[], int n);
double medirTempoMerge(int vetor[], int n);
double medirTempoQuick(int vetor[], int n);

#endif
```

Anexo B – Código-fonte main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "ordenadores.h"

void copiarVetor(int origem[], int destino[], int n) {
    for (int i = 0; i < n; i++) {
        destino[i] = origem[i];
    }
}

void gerarVetorAleatorio(int vetor[], int n) {
    for (int i = 0; i < n; i++) {
        vetor[i] = rand() % 100000;
    }
}

int main() {
    srand(time(NULL));
```

```
int tamanhos[] = {100, 300, 500, 1000, 10000};
int quantidadeTamanhos = 5;

FILE *arquivo = fopen("resultados/resultados.csv", "w");

if (arquivo == NULL) {
    printf("Erro ao criar o arquivo resultados/resultados.csv.\n");
    return 1;
}

fprintf(arquivo, "Tamanho;Bubble Sort (ms);Merge Sort (ms);Quick Sort
(ms)\n");

for (int i = 0; i < quantidadeTamanhos; i++) {
    int n = tamanhos[i];

    int *vetorOriginal = (int *) malloc(n * sizeof(int));
    int *vetorBubble = (int *) malloc(n * sizeof(int));
    int *vetorMerge = (int *) malloc(n * sizeof(int));
    int *vetorQuick = (int *) malloc(n * sizeof(int));

    if (
        vetorOriginal == NULL ||
        vetorBubble == NULL ||
        vetorMerge == NULL ||
        vetorQuick == NULL
    ) {
        printf("Erro ao alocar memoria.\n");
        fclose(arquivo);
        return 1;
    }

    gerarVetorAleatorio(vetorOriginal, n);

    copiarVetor(vetorOriginal, vetorBubble, n);
    copiarVetor(vetorOriginal, vetorMerge, n);
    copiarVetor(vetorOriginal, vetorQuick, n);

    double tempoBubble = medirTempoBubble(vetorBubble, n);
    double tempoMerge = medirTempoMerge(vetorMerge, n);
    double tempoQuick = medirTempoQuick(vetorQuick, n);
```



```
    fprintf(arquivo, "%d;%.0f;%.0f;%.0f\n", n, tempoBubble, tempoMerge,
tempoQuick);

    free(vetorOriginal);
    free(vetorBubble);
    free(vetorMerge);
    free(vetorQuick);
}

fclose(arquivo);

printf("\nArquivo resultados/resultados.csv criado com sucesso!\n");

return 0;
}
```

Anexo C – Código-fonte bubble-sort.c

```
#include <time.h>
#include "../ordenadores.h"

void bubbleSort(int vetor[], int n) {
    int temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (vetor[j] > vetor[j + 1]) {
                temp = vetor[j];
                vetor[j] = vetor[j + 1];
                vetor[j + 1] = temp;
            }
        }
    }
}

double medirTempoBubble(int vetor[], int n) {
    clock_t inicio, fim;

    inicio = clock();
    bubbleSort(vetor, n);
    fim = clock();

    return ((double)(fim - inicio) / CLOCKS_PER_SEC) * 1000.0;
}
```

```
}
```

Anexo D – Código-fonte merge-sort.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "../ordenadores.h"

static void merge(int vetor[], int inicio, int meio, int fim) {
    int n1 = meio - inicio + 1;
    int n2 = fim - meio;

    int *esquerda = (int *) malloc(n1 * sizeof(int));
    int *direita = (int *) malloc(n2 * sizeof(int));

    if (esquerda == NULL || direita == NULL) {
        printf("Erro ao alocar memoria no Merge Sort.\n");
        exit(1);
    }

    for (int i = 0; i < n1; i++) {
        esquerda[i] = vetor[inicio + i];
    }

    for (int j = 0; j < n2; j++) {
        direita[j] = vetor[meio + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = inicio;

    while (i < n1 && j < n2) {
        if (esquerda[i] <= direita[j]) {
            vetor[k] = esquerda[i];
            i++;
        } else {
            vetor[k] = direita[j];
            j++;
        }
        k++;
    }
}
```

```
while (i < n1) {
    vetor[k] = esquerda[i];
    i++;
    k++;
}

while (j < n2) {
    vetor[k] = direita[j];
    j++;
    k++;
}

free(esquerda);
free(direita);
}

void mergeSort(int vetor[], int inicio, int fim) {
    if (inicio < fim) {
        int meio = inicio + (fim - inicio) / 2;

        mergeSort(vetor, inicio, meio);
        mergeSort(vetor, meio + 1, fim);

        merge(vetor, inicio, meio, fim);
    }
}

double medirTempoMerge(int vetor[], int n) {
    clock_t inicio, fim;

    inicio = clock();
    mergeSort(vetor, 0, n - 1);
    fim = clock();

    return ((double)(fim - inicio) / CLOCKS_PER_SEC) * 1000.0;
}
```

Anexo E – Código-fonte quick-sort.c

```
#include <time.h>
#include "../ordenadores.h"
```

```
static int particionar(int vetor[], int baixo, int alto) {
    int pivo = vetor[alto];
    int i = baixo - 1;
    int temp;

    for (int j = baixo; j < alto; j++) {
        if (vetor[j] <= pivo) {
            i++;

            temp = vetor[i];
            vetor[i] = vetor[j];
            vetor[j] = temp;
        }
    }

    temp = vetor[i + 1];
    vetor[i + 1] = vetor[alto];
    vetor[alto] = temp;

    return i + 1;
}

void quickSort(int vetor[], int baixo, int alto) {
    if (baixo < alto) {
        int indicePivo = particionar(vetor, baixo, alto);

        quickSort(vetor, baixo, indicePivo - 1);
        quickSort(vetor, indicePivo + 1, alto);
    }
}

double medirTempoQuick(int vetor[], int n) {
    clock_t inicio, fim;

    inicio = clock();
    quickSort(vetor, 0, n - 1);
    fim = clock();

    return ((double)(fim - inicio) / CLOCKS_PER_SEC) * 1000.0;
}
```